

TRABAJO EXAMEN FINAL - PROGRAMACIÓN II

Head Soccer

Alumno: Ignacio Rodriguez

Docente: Santiago Fazzini

Fecha de Presentación: 9 de Diciembre 2025

1. Descripción del Software

Presentación

Head Soccer es un videojuego de deportes arcade "1 vs 1" desarrollado en C++ utilizando la biblioteca multimedia **Raylib**. El proyecto recrea la experiencia de partidos de fútbol rápidos y dinámicos, donde dos jugadores compiten localmente controlando avatares ("cabezones") con habilidades de salto y pateo.

Detalles del Desarrollo y Arquitectura

El software fue construido bajo el paradigma de **Programación Orientada a Objetos (POO)**, priorizando un diseño modular y mantenible mediante el uso de **Composición** sobre herencia compleja.

Los pilares técnicos del desarrollo incluyen:

- **Máquina de Estados (StateManager):** El flujo del programa (Menú -> Selección -> Juego -> Gol -> Pausa) está controlado por una máquina de estados finita, lo que permite transiciones limpias y evita el "código espagueti" en el bucle principal.
- **Motor de Física Personalizado (Clase Estática Física):** Se implementó un motor de física propio en la clase Física. Este motor calcula colisiones, resuelve rebotes, aplica gravedad y fricción. Permite la **configuración dinámica de variables** (gravedad lunar, hielo, etc.) en tiempo de ejecución a través de un único llamado a `Fisica::setConfiguracion`.
- **Gestión de Assets (AssetManager):** Un AssetManager centralizado se encarga de la carga y descarga de texturas según el modo de juego seleccionado, optimizando el uso de memoria VRAM.
- **Arquitectura Orientada a Datos (ConfigModo):** Los modos de juego son estructuras de datos (ConfigModo) que inyectan reglas al motor físico y rutas al gestor de recursos, facilitando la creación de nuevos escenarios sin modificar la lógica central.

Bibliotecas de Terceros

- **Raylib (v5.0):** Utilizada para la capa de abstracción de hardware, manejo de ventana, input (teclado/mouse), renderizado de texturas y matemáticas vectoriales básicas (Vector2, CheckCollision...).
- **Tau (Microframework):** Utilizado para la ejecución de pruebas unitarias (Test Driven Development) automatizadas.

2. Manual de Uso

Objetivo: Anotar más goles que el rival golpeando la pelota hacia el arco contrario.

Controles

Función	Jugador 1 (Izquierda)	Jugador 2 (Derecha)
Mover Izquierda/Derecha	A / D	Flecha Izquierda / Flecha Derecha
Saltar	W	Flecha Arriba
Patear (Impulso fuerte)	S	Flecha Abajo
Pausar / Volver	ESC	ESC

3. Ficha Técnica de Componentes (Catálogo de Clases)

Archivo	Clase/Struct	Tipo	Responsabilidad Principal	Cita de Código Clave
GameManager.h/cpp	GameManager	Clase Principal (Composición)	Orquesta todo el juego, maneja el ciclo de vida de Jugador, Pelota y Cancha. Controla el <i>buclePrincipal</i> .	if (ptrPelota) delete ptrPelota;

Fisica.h/.cpp	Fisica	Clase Estática (static)	Contiene las reglas del mundo: gravedad, fricción, rebote y fuerzas de impacto. Permite la configuración de modos.	static float gravedad; void setConfiguracion(...)
Jugador.h/.cpp	Jugador	Entidad Dinámica	Controla el movimiento, salto y pateo del personaje. Lee la velocidad desde Fisica::velocidadJugador.	velocidad.x *= Fisica::rozamiento;
Pelota.h/.cpp	Pelota	Entidad Dinámica	Maneja su posición, velocidad y rotación. Contiene la lógica de límites de pantalla y rebote en el piso.	if (posicion.y >= Y_PISO_PELOTA - radio)
Cancha.h/.cpp	Cancha	Contenedor Estático	Almacena los dos objetos Arco y la textura de fondo.	this->arcolzq = Arco(ARCO_IZQ_ZONA, ...);
Arco.h/.cpp	Arco	Entidad Geométrica	Define la geometría del arco (postes, travesaño) y la	bool checkGol(const Pelota& p) const

			zona de gol invisible.	
AssetManager.h.cpp	AssetManager	Gestor de Recursos	Centraliza la carga y descarga de todas las texturas de los modos de juego.	UnloadTexture(texPelota); texJugador1 = LoadTexture(pathJ1.c_str());
SelectorModos.h.cpp	SelectorModos	Menú Interactivo	Gestiona la lista de ConfigModo. Permite la navegación y la selección de modos.	std::vector<ConfigModo> modos;
SelectorModos.h	ConfigModo	Struct (Datos)	Contenedor de datos del modo de juego (rutas de texturas y 7 parámetros de física).	float gravedad; std::string texCancha;
Estados.h	StateManager	Clase Auxiliar	Almacena el estado actual del juego (EstadoJuego) y provee métodos para consultarlo.	bool esMenu() const
Constantes.h	(No es clase)	Datos Globales	Define todas las constantes del juego: dimensiones de pantalla y coordenadas de los arcos.	const int ANCHO_PANTALLA = 1024;

4. Tests de Unidad (TDD)

Implementación y Enfoque

Para garantizar la robustez del modelo lógico, se diseñaron pruebas unitarias automatizadas utilizando el microframework **Tau**. El enfoque TDD se concentró en las mecánicas críticas del juego, validando la integridad de las estructuras de datos y el motor físico.

El código fuente de estos tests se encuentra adjunto en el archivo `src/test.cpp`.

Casos de Prueba Implementados

Suite de Tests	Objetivo	Tipo de Asersión
Modelo.Arco	Verificar la detección de goles en límites y centro.	CHECK
Fisica.Gravedad	Validar la aceleración correcta y la simulación de caída en el eje Y.	CHECK, REQUIRE
Fisica.Friccion	Verificar que la velocidad horizontal de la pelota se reduzca (fricción) correctamente.	CHECK
Fisica.Config	Asegurar que la clase estática Fisica almacene correctamente los 7 parámetros de modo de juego.	CHECK
Integracion.Colisiones	Verificar que Fisica::resolverColisiones detecte la superposición y desplace la pelota fuera de los jugadores.	CHECK
Integracion.Pateo	Comprobar que el impulso aplicado por el jugador (fuerzaPateoX/Y) sea significativamente mayor	CHECK

	que la velocidad de rebote normal.	
Constantes.Integridad	Validar la lógica y los límites de la pantalla (ANCHO_PANTALLA, coordenadas de arcos).	CHECK